Objectifs de la feuille

- Formulaire dédié au modèle
- La vue générique de formulaire
- La vue générique de création
- La vue générique de mise à jour
- La vue générique de suppression
- Tests des formulaires

Les formulaires dédiés au modèle

Maintenant que nous savons comment créer des formulaires dans DJANGO, la prochaine étape est de créer un formulaire qui permettra aux utilisateurs de créer un nouvel objet Produit. On pourrait penser qu'on commencerait par définir une autre classe de formulaire et, pour chaque champ de notre modèle, définir un champ de formulaire correspondant et vous pourriez le faire !

Même si nous construisons deux choses distinctes, un modèle et un formulaire, il semble y avoir beaucoup d'informations dupliquées entre les deux. Nous avons déjà défini chaque champ dans le modèle, en spécifiant les types de données (string, integer) et les règles (par exemple max_length). Pourquoi devrions-nous les définir également dans le formulaire ? Ne serait-il pas formidable de pouvoir laisser le modèle définir le formulaire ? C'est exactement ce à quoi servent les ModelForm.

Dans monApp/forms.py, nous allons définir une nouvelle classe qui hérite de django.forms.ModelForm :

```
class ProduitForm(forms.ModelForm):
    class Meta:
        model = Produit
        fields = '__all__'
```

La nouvelle classe contient une classe imbriquée, Meta, qui spécifie le modèle pour lequel ce formulaire sera utilisé, et les champs de ce modèle à inclure dans ce formulaire (dans notre cas, tous les champs). Maintenant, utilisons notre ProductForm dans la vue ProductCreate en le passant au modèle :

```
def ProduitCreate(request):

    form = ProduitForm()
    titremenu = "Création d'un nouveau produit"
    return render(request, "monApp/create_produit.html", {'titremenu':titremenu,'form': form})
```

Prenons soin de mettre à jour le fichier monApp/urls.py avec la nouvelle url :

```
path("produit/",views.ProduitCreate , name="creationProduit"),
```

Et ensuite nous générons le formulaire dans le template create_produit.html :

```
{% extends 'monApp/base.html' %}
{% block menu %}
<h1>{{titremenu}}</h1>
{% endblock %}
{% block contenu %}
<form action="" method="post" novalidate>
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit" class="btn btn-success mt-2">Créer</button>
</form>
{% endblock %}
```

Jetez un coup d'œil à notre page dans le navigateur à l'adresse http://127.0.0.1:8000/monApp/produit/. Nous venons de générer un formulaire pour notre modèle, sans avoir à définir aucun champ de formulaire, nous avons laissé le modèle le faire pour nous ! Et voici la meilleure partie : si nous modifions les champs du modèle, le formulaire est automatiquement mis à jour. Mais nous n'avons pas encore fini : nous devons mettre à jour notre vue pour qu'elle crée un objet Produit à partir des valeurs que nous soumettons dans le formulaire.

Nous allons suivre le même schéma que pour le formulaire de « Contact ». Récapitulons le cycle de vie de ce modèle de vue, en style « pseudocode » :

- La demande est transmise à la fonction de vue.
    - S'il s'agit d'une demande GET :
        - Il doit s'agir de la première requête sur cette page et l'utilisateur s'attend à voir un formulaire vierge, prêt à être rempli. Nous créons donc une nouvelle instance vide du formulaire et nous la stockons dans une variable « form ».
    - S'il s'agit d'une demande POST :
        - Il ne s'agit pas de la première demande, mais d'une demande ultérieure à cette page, accompagnée de certaines valeurs de formulaire que l'utilisateur aura soumises. Nous créons donc une instance du formulaire et nous la remplissons avec les valeurs envoyées dans la requête POST. C'est stocké dans une variable « form ».
        - Si la validation du formulaire se déroule avec succès :
            - Nous effectuons l'action, qu'il s'agisse d'envoyer un e-mail, de créer un objet ou de toute autre action.
            - Nous redirigeons l'utilisateur vers une autre vue, peut-être une page de confirmation ou une autre page qui indique que l'action a réussi. Nous arrêtons d'exécuter cette fonction de vue à ce stade. Cependant...
        - Si la validation du formulaire n'a PAS réussi :
            - Le processus de validation a maintenant ajouté des messages d'erreur dans les champs concernés.
            - Nous autorisons l'exécution de cette vue pour continuer à l'étape suivante ci-dessous.

Nous avons maintenant un «formulaire» qui est soit

- nouveau et vide,
- rempli avec des valeurs précédemment soumises et des messages d'erreur pertinents.

Dans les deux cas, nous passons « form » au modèle pour qu'il puisse être affiché dans le navigateur. Voici à quoi cela ressemble en Python, dans notre vue ProduitCreate. Notez que nous traitons d'abord le cas POST dans l'instruction if et ensuite le cas GET dans l'instruction else :

```
def ProduitCreate(request):
    titremenu = "Ajout d'un nouveau produit"
    if request.method == 'POST':
        form = ProduitForm(request.POST)
        if form.is_valid():
            prdt = form.save()
            return redirect('dtl-prdt', prdt.refProd)
    else:
        form = ProduitForm()

    return render(request, "monApp/create_produit.html", {'titremenu':titremenu,'form': form})
```

Notez que la méthode form.save() ne fait pas qu'enregistrer le nouvel objet dans la base de données : elle renvoie également cet objet, ce qui signifie que nous pouvons l'utiliser à l'étape suivante, où nous redirigeons immédiatement vers le produit nouvellement créé.

Maintenant nous pouvons remplir le formulaire et nous sommes alors redirigés vers la page détaillée du produit que nous venons de créer. Notre nouvelle page fonctionne, mais elle ne sert à rien si nos utilisateurs ne peuvent pas la trouver. Nous devons créer un lien vers elle quelque part. Et si on établissait un lien à partir de la vue en liste de Produit, dans le template list_produit.html:

```
<a href="{% url 'creationProduit' %}">Créer un nouveau produit</a>
```

Les ModelForms sont un outil puissant pour créer des modèles rapidement et facilement.C'est formidable que DJANGO puisse générer un formulaire pour nous, mais que faire si le formulaire par défaut ne répond pas tout à fait à nos besoins ? Vous pouvez toujours créer un formulaire à partir de zéro, comme nous l'avons fait avec le ContactUsForm et l'utiliser. Mais il est également possible de personnaliser un ModelForm.

Disons que nous voulons que les utilisateurs puissent créer un nouveau produit, mais nous ne voulons pas qu'ils puissent modifier les champs categorie et status, nous voulons que seuls les administrateurs puissent modifier ces champs (ce qu'ils peuvent faire dans le site d'administration). Nous pouvons les exclure du formulaire, comme ceci :

```
class ProduitForm(forms.ModelForm):
    class Meta:
        model = Produit
        #fields = '__all__'
        exclude = ('categorie', 'status')
```

Et alors ces champs n'apparaîtront pas dans le formulaire. Le formulaire fonctionnera toujours et nous pourrons créer un nouvel objet Produit sans aucune erreur, même si nous avons laissé ces champs de côté, parce que status et categorie autorise les valeurs NULL avec null=True dans notre modèle.

Si vous voulez exclure certains des autres champs (non nullables) du formulaire, vous devrez d'abord leur donner une valeur par défaut, ou les rendre nullables, dans monApp/models.py. Vous devrez également effectuer et exécuter une migration pour mettre à jour le schéma de la base de données, car les valeurs par défaut et les valeurs NULL sont configurées dans la base de données elle-même.

La vue générique de formulaire DJANGO

FormView traite un formulaire qui n'est pas forcément lié à un modèle, comme notre formulaire de contact. Le code est plus concis, la gestion du formulaire est prise en charge par la classe parente, et nous n'avons plus besoin de tester explicitement request.method == "POST" ni d'instancier le formulaire dans la vue. FormView s'en charge automatiquement. La vue ContactView basée sur la classe devient :

```
class ContactView(FormView):
    template_name = "monApp/page_home.html"
    form_class = ContactUsForm
    success_url = reverse_lazy("email-sent")

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["message"] = "Contact Us"
        return context

    def form_valid(self, form):
        send_mail(
        subject=f'Message from {form.cleaned_data["name"] or "anonyme"} via MonProjet Contact Us form',
        message=form.cleaned_data["message"],
        from_email=form.cleaned_data["email"],
        recipient_list=["admin@monprojet.com"],
        )
        return super().form_valid(form)
```

Dans monApp/urls.py, l'url change : path("contact/",views.ContactView.as_view() , name="contact"),

La vue générique de création DJANGO

Est-ce qu'il nous arrivera de créer manuellement un formulaire pour un modèle, ou est-ce que nous utiliserons toujours un ModelForm ? Vous devriez très rarement avoir à créer manuellement un formulaire pour un modèle à partir de zéro, bien que rien ne vous empêche de le faire. Il faudrait alors que le modèle et le formulaire soient synchronisés l'un avec l'autre. L'utilisation d'un ModelForm dans le premier cas est logique, car vous gardez les choses DRY et il y a moins de code à maintenir.

Là encore, vous pouvez passer par la vue générique CreateView que fournit le framework DJANGO. Les changements s'opèrent dans le fichier views.py en transformant notre vue basée sur une fonction en une vue basée sur une classe qui hérite de CreateView :

```
from django.forms import BaseModelForm

class ProduitCreateView(CreateView):
        model = Produit
        form_class=ProduitForm
        template_name = "monApp/create_produit.html"

        def get_context_data(self,**kwargs):
                context = super(ProduitCreateView, self).get_context_data(**kwargs)
                context['titremenu'] = "Ajout d'un nouveau produit"
                return context

        def form_valid(self, form: BaseModelForm) -> HttpResponse:
                prdt = form.save()
                return redirect('detailProduit', prdt.refProd)
```

Et n'oublions pas la modification dans monApp/urls.py :

```
path("produit/",views.ProduitCreateView.as_view(), name="creationProduit"),
```

La vue générique de mise à jour DJANGO

Maintenant qu'on a créé les objets du modèle, on doit être en mesure de les mettre à jour. Il est temps de choisir un motif d'URL pour notre prochaine vue, qui servira à mettre à jour un produit existant. Jusqu'à présent, nous avons :

- produits/ pour la liste des produits ;
- produit/1/ pour le produit 1, «produit/2/» pour le produit 2, etc. ;
- produit/ pour créer un produit.

Le problème, c'est qu'il faut savoir quel produit changer. Mettons aussi cette information dans l'URL. Ce sera donc «produit/1/change/» pour mettre à jour le produit 1, «produit/2/change/» pour mettre à jour le produit 2 et ainsi de suite. Là encore, vous pouvez utiliser le verbe «update» ou tout autre mot de votre choix, à la place de «change». Pourquoi pas «produit/change/1/», avec l'identifiant à la fin de l'URL ? Ça marcherait aussi ! Lorsque nous choisissons nos chemins, nous pouvons choisir la structure parent-enfant qui nous semble la plus logique. J'aime imaginer que la page de modification est un «enfant» de la page de détail du produit. Ainsi, si la page de détail se trouve à l'adresse «produit/1/», je place la page des modifications pour ce produit à l'adresse «produit/1/change/». Il faut modifier le fichier urls.py en conséquence :

```
path("produit/<pk>/update/",views.ProduitUpdateView.as_view(), name="modificationProduit"),
```

Vous devriez maintenant connaître la marche à suivre, il faut une nouvelle vue et un gabarit de base pour la mise à jour d'un produit. Plutôt que d'écrire une fonction pour la vue, je crée une nouvelle classe héritant de la vue générique UpdateView fournie par DJANGO.

```
class ProduitUpdateView(UpdateView):
    model = Produit
    form_class=ProduitForm
    template_name = "monApp/update_produit.html"

    def get_context_data(self,**kwargs):
        context = super(ProduitUpdateView, self).get_context_data(**kwargs)
        context['titremenu'] = "Modification du produit"
        return context

    def form_valid(self, form: BaseModelForm) -> HttpResponse:
        prdt = form.save()
        return redirect('detailProduit', prdt.refProd)
```

Et le petit template qui va bien dans le fichier update_produit.html :

```
{% extends 'monApp/base.html' %}

{% block menu %}
<h1>{{titremenu}}</h1>
{% endblock %}

{% block contenu %}
<form action="" method="post" >
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit" class="btn btn-success mt-2">Modifier</button>
</form>
{% endblock %}
```

Ce qui est bien avec les formulaires dans DJANGO, c'est qu'il n'est pas nécessaire d'en créer un nouveau spécifiquement pour l'opération de mise à jour : nous pouvons réutiliser le formulaire que nous utilisons déjà pour l'opération de création. Après tout, nous ne faisons que mettre à jour les mêmes champs que ceux avec lesquels nous avons travaillé précédemment.

Il faut enfin créer un lien vers la page de mise à jour. Il existe deux endroits à partir desquels nous pourrions créer un lien vers une page de «mise à jour» du produit. Mettons en œuvre les deux possibilités :

- nous pouvons établir un lien à partir de la vue en liste produit
- nous pouvons établir un lien à partir de la vue détaillée du produit

Dans les deux cas, c'est le même code : c'est à vous de la mettre au bon endroit :

```
<a href="{% url 'modificationProduit' prdt.refProd %}">modifier</a>
```

La vue générique de suppression DJANGO

L'opération de suppression est une opération à laquelle il faut faire particulièrement attention. Dès que nous supprimons un objet de la base de données, il disparaît définitivement, il n'y a pas de fonction d'annulation. Nous devons donc être sûrs que c'est ce que l'utilisateur avait vraiment l'intention de faire et que ce n'est pas un accident !

Pour cette raison, les interfaces CRUD exigent souvent deux étapes pour supprimer un objet. Par exemple, dans l'administration de Django, la première étape est une sorte de bouton de suppression. En cliquant sur le bouton, au lieu d'effectuer immédiatement l'opération de suppression, vous êtes redirigé vers une page de confirmation, donnant à l'utilisateur la possibilité de changer d'avis.

Ce n'est qu'après avoir cliqué sur «Yes, I'm sure» (Oui, je suis sûr) que l'opération de suppression sera effectuée dans la base de données. Implémentons quelque chose de similaire dans notre interface CRUD.

Nous allons avoir besoin d'une autre vue pour cette page de confirmation, ce qui signifie que nous avons également besoin d'un nouveau modèle d'URL Suivons le modèle que nous avons établi au chapitre précédent, la page de mise à jour du produit 1 se situait au chemin : «produit/1/update/». Donc la page de confirmation pour la suppression du groupe 1 sera : «produit/1/delete/».

Allez-y et créez le modèle d'URL, la vue et le gabarit de base pour notre page de confirmation de suppression. Le modèle a juste besoin d'un en-tête pour que nous puissions tester qu'il se charge correctement. Etablissons également un lien entre la page détaillée du produit et la page de confirmation de la suppression. Nous avons déjà procédé de la sorte pour la page de mise à jour dans le chapitre précédent, alors suivez la même méthode et reportez-vous à ce chapitre si vous avez besoin d'un rappel. Je vous laisse le choix de l'implémentation entre une fonction ou une classe pour la vue, mais sachez que DJANGO fournit une classe DeleteView…

Lorsqu'un utilisateur clique sur la page de suppression, la première chose à faire est de confirmer quel objet nous sommes sur le point de supprimer. Nous pouvons le faire en affichant l'un des champs de l'objet. Affichons donc le nom du produit à l'utilisateur, afin qu'il puisse vérifier qu'il s'agit bien de celui qu'il voulait supprimer.

Afin de supprimer réellement l'objet, nous allons une fois de plus utiliser un formulaire et une requête HTTP POST. Mais ce formulaire est différent du ModelForm que nous avons utilisé jusqu'à présent : il ne va pas envoyer de valeurs pour les champs du modèle, parce que nous n'avons besoin de changer aucun de ces champs, nous voulons juste supprimer l'objet dans sa totalité.

Le formulaire est si simple que nous pouvons le coder directement dans le template delete_produit.html sans créer de nouvelle classe de formulaire :

```
{% extends 'monApp/base.html' %}

{% block contenu %}
<h1> Suppression d'un produit </h1>
<form action="" method="post" >
        {% csrf_token %}
        <p>Etes-vous sûr de vouloir supprimer le produit  {{ object.intituleProd }} ?</p>
        <button type="submit" class="btn btn-success mt-2">Confirmer</button>
</form>

{% endblock %}
```

Le formulaire ne comporte qu'une seule entrée : le bouton d'envoi (deux si vous comptez le jeton CSRF). Nous n'avons besoin de rien de plus.

Mais comment le serveur saura-t-il quel produit supprimer ? Ne devrions-nous pas au moins envoyer l'id du produit ? Nous envoyons déjà l'identifiant du groupe dans l'URL ! Si nous sommes à l'url «/produit/1/delete/» et que nous soumettons le formulaire, la requête POST renvoi à la même URL. L'identifiant est transmis à la vue et c'est ainsi que la vue sait quel produit supprimer.

Ensuite dans la vue, nous utilisons le modèle que nous avons déjà utilisé précédemment. Nous redirigeons ensuite l'utilisateur vers la liste des produit, où il verra que le produit qu'il a supprimé n'apparaît plus dans la liste. Remarquez que la validation du formulaire n'est pas nécessaire ici, car il n'y a pas de données à valider (pas de méthode form_valid)

```
class ProduitDeleteView(DeleteView):
        model = Produit
        template_name = "monApp/delete_produit.html"
        success_url = reverse_lazy('listeProduits')
```

Avec cette vue, notre url est donc :

```
path("produit/<pk>/delete/",views.ProduitDeleteView.as_view(), name="suppressionProduit"),
```

Test des formulaires et des nouvelles vues

Pour la vue de création, il faut faire comme précédemment, c'est à dire, tester l'url de la page et son contenu. Vous savez faire puisque c'est ce que nous faisons depuis le début pour ces vues utilisant la méthode GET. Mais la nouveauté réside dans le fait que la vue de création utilise aussi la méthode POST suite à la validation du formulaire. Notre classe ProduitCreateViewTest est enrichie de cette fonction :

```
def test_categorie_create_view_post_valid(self):
        data = { "intituleProd": "ProduitPourTestCreation",
                 "prixUnitaireProd":100.0,
                 "dateFabProd":date(2023, 1, 1),
                 "categorie":"",
                 "statut":"" }
        #simulation soumission formulaire
        response = self.client.post(reverse('creationProduit'), data)
        # Vérifie la redirection après la création
        self.assertEqual(response.status_code, 302)
        # Vérifie qu'un objet a été créé
        self.assertEqual(Produit.objects.count(), 1)
        # Vérifie la valeur de l'objet créé
        self.assertEqual(Produit.objects.last().intituleProd, 'ProduitPourTestCreation')
```

C'est à vous de jouer !

Afin de maintenir votre taux de couverture de tests, mettez en œuvre les tests unitaires sur les vues de modification et de suppression d'un produit (méthodes GET et POST)

Mais ce n'est pas tout. Il faut tester aussi individuellement le formulaire ProduitForm. En effet, lorsque nous regardons le modèle de Produit, plusieurs conclusions doivent être tirées :

- le champ intituleProd est obligatoire et faire moins de 200 caractères
- le champ prixUnitaireProd est obligatoire et respecter 10 digits dont 2 decimals
- les champs categorie et statut peuvent être nuls (donc, nous le testerons pas)

Dans le fichier tests/test_Form_Produit.py que vous allez créer, écrivez ceci pour tester le champ intituleProd (je vous laisse faire de même pour prixUnitaireProd)

```
class ProduitFormTest(TestCase):

    def setUp() :
        self.produit = Produit.objects.create(intituleProd="ProduitPourTestFormulaire",
        prixUnitaireProd=10.0,dateFabProd= date(2023, 1, 1),categorie=None,statut=None)

    # test du champ intituleProd trop long
    def test_form_valid_data_too_long(self):

        data = {"intituleProd": self.produit.intituleProd*10,
        "prixUnitaireProd":self.produit.prixUnitaireProd,
        "dateFabProd":self.produit.dateFabProd,
        }

        form = ProduitForm(data)
        self.assertFalse(form.is_valid()) # Le formulaire doit être invalide
        self.assertIn('intituleProd', form.errors) # Le champ 'intituleProd' doit contenir une erreur

        self.assertEqual(form.errors['intituleProd'], ['Assurez-vous que cette valeur comporte au plus 200 caractères
        (actuellement 250).'])


    # test du champ intituleProd manquant
    def test_form_valid_data_missed(self):

        data = {"intituleProd": "",
        "prixUnitaireProd":self.produit.prixUnitaireProd,
        "dateFabProd":self.produit.dateFabProd,
        }
        form = ProduitForm(data)
        self.assertFalse(form.is_valid()) # Le formulaire doit être invalide
        self.assertIn('intituleProd', form.errors) # Le champ 'intituleProd' doit contenir une erreur
        self.assertEqual(form.errors['intituleProd'], ['Ce champ est obligatoire.'])
```

Il ne faut oublier tester aussi quand le formulaire est valide et le produit bien pris en compte :

```
    def test_form_valid_and_save(self):

        data = {"intituleProd": self.produit.intituleProd,
        "prixUnitaireProd":self.produit.prixUnitaireProd,
        "dateFabProd":self.produit.dateFabProd,
        }

        form = ProduitForm(data)
        self.assertTrue(form.is_valid())
        prdt = form.save()
        self.assertEqual(prdt.intituleProd, 'ProduitPourTestFormulaire')
```

C'est à vous de jouer !

Vous avez du pain sur la planche. Vous allez faire le même travail pour Categorie, Statut et Rayon (ne criez pas, il y a juste un id et un champ pour ces trois là, vous avez fait le plus long avec Produit).

Au programme :

- Ajouter une page Créer (modèle d'URL, vue et gabarit) pour chacun des éléments;
- Ajouter un formulaire de type ModelForm et l'utiliser dans la nouvelle page pour créer les objets ;
- Lier la liste des objets à la page de création d'un objet ;
- Ajouter une page de «Mise à jour» (modèle d'URL, vue et gabarit) pour chacun des éléments
- Utiliser le Form existant dans cette page pour mettre à jour les objets ;
- Lier les pages de liste des objets et de détails d'un objet vers la page de mise à jour.
- Ajouter une page de suppression (modèle d'URL, vue et gabarit) pour chacun des éléments
- Inclure un formulaire de suppression dans le gabarit
- Rediriger l'utilisateur vers la liste des objets après l'opération de suppression ;
- Lier la page détaillée des objets à la page de suppression.
- Tester unitairement les nouveaux formulaires pour chacun des éléments
- Tester unitairement les nouvelles vues de création, modification et suppression pour chacun des éléments

Voilà. Vive le CRUD ! Ce n'est que du « copier-coller » (ou pas) mais avec de la rigueur, de la discipline et de la concentration, vous allez y arriver. Après tout, vous êtes en BUT3 et vous devez comprendre ce que l'on attend de vous tel un professionnel digne de ce nom : concevoir, développer et valider votre application métier.